**COMPARING TEMPORAL LOGIC REWARD SHAPING APPROACHES IN REINFORCEMENT LEARNING**

By

Braeden Treutel

APPROVED:

Graduate Committee:

_____Zhijiang Dong_____

Supervisor Dr. professor's name (Computer Science)

_____Arpan Sainju_____

Dr. professor's name (Computer Science)

_____Joshua Phillips_____

Dr. professor's name (Computer Science or other department)

_____Medha Sarkar_____

Dr. chair's name, Chairperson Computer Science Department

_____David Butler_____

Dr. dean's name, Dean of the College of Graduate Studies

# COMPARING TEMPORAL LOGIC REWARD SHAPING APPROACHES IN REINFORCEMENT LEARNING

By

Braeden Treutel

A thesis submitted in partial fulfillment

of the requirements for the degree of

MASTER OF SCIENCE

in

Computer Science

Middle Tennessee State University

December 2027

**ABSTRACT**

Temporal logic specifications are increasingly used to provide safety and task guarantees for reinforcement learning agents, yet most existing approaches assume infinite-horizon objectives. We empirically compare infinite-horizon automaton-based methods (LTL) with finite-horizon robustness-based formulations (TLTL) across three MiniGrid benchmark environments of increasing complexity, evaluating specification satisfaction probability, mean time to satisfaction, and sample efficiency across 30 independent runs. On simple tasks, all three methods performed comparably. On the sequential DoorKey-8x8 task, TLTL achieved 97.4% satisfaction vs 6.4% (LTL) and 2.0% (Baseline) ($p<0.001$), demonstrating that finite-horizon robustness rewards provide significantly more effective learning signals for complex ordered subtask problems, while infinite-horizon methods may offer stronger theoretical guarantees.

## TABLE OF CONTENTS

LIST OF TABLES ..... v

LIST OF FIGURES ..... vi

LIST OF SYMBOLS AND ABBREVIATIONS ..... vii

Chapter

I. INTRODUCTION ..... 1

II. BACKGROUND ..... 3

III. METHODS ..... 7

IV. RESULTS ..... 12

V. DISCUSSION ..... 17

VI. CONCLUSION ..... 20

BIBLIOGRAPHY

## LIST OF TABLES

Table 1 – Mean ± standard deviation of specification satisfaction rate..... 12

Table 2 – Mean ± standard deviation of episode length across successful episodes.... 14

Table 3 – Mean ± standard deviation of timesteps to 80% episode length reduction .. 14

## LIST OF FIGURES

Figure 1 – Overview of experimental pipeline. ..... 7

Figure 2 – MiniGrid environments and corresponding temporal specifications ..... 8

Figure 3 – Bar plots of evaluation metrics across all runs ..... 12

Figure 4 – Specification satisfaction rate boxplots ..... 13

Figure 5 – Episode length learning curves averaged over 30 runs ..... 15

## LIST OF SYMBOLS AND ABBREVIATIONS

RL – Reinforcement Learning

LTL – Linear Temporal Logic

TLTL – Truncated Linear Temporal Logic

PPO – Proximal Policy Optimization

MDP – Markov Decision Process

STL – Signal Temporal Logic

# CHAPTER I

# INTRODUCTION

Reinforcement learning (RL) is a powerful tool for sequential decision making in complex environments, where an agent learns an optimal policy through interaction with its environment using reward feedback [9]. Despite its successes, classical RL is a black box learning method that optimizes cumulative reward without providing formal guarantees of safety or logical task satisfaction. To address this limitation, recent work incorporates temporal logic specifications into the learning process to enforce behavioral constraints with formal guarantees [4]. Specifications can be formulated over either infinite time horizons, typically enforced through automaton constructions, or finite time horizons, using robustness measures that provide graded, continuous feedback. Despite growing interest in both approaches, direct empirical comparisons between finite-horizon and infinite-horizon formulations across identical environments and learning algorithms remain limited.

This work addresses that gap by empirically comparing infinite-horizon automaton-based LTL with finite-horizon robustness-based TLTL across three Gymnasium MiniGrid environments of increasing complexity. Both approaches are integrated into an identical PPO training framework to isolate the effect of temporal logic formulation. Performance is evaluated using specification satisfaction probability, mean time to satisfaction, and sample efficiency across 30 independent runs.

Results demonstrate that on simple tasks, all three methods (vanilla PPO baseline, LTL, and TLTL) perform comparably. On the sequential DoorKey-8x8 task, TLTL significantly outperforms both alternatives, achieving 94.8% specification satisfaction

compared to 7.2% for LTL and 0.6% for the baseline ($p<0.001$). These findings suggest that finite-horizon robustness-based specifications provide more effective learning signals for complex ordered subtask problems, while infinite-horizon methods may offer stronger theoretical guarantees at the cost of learning efficiency.

# CHAPTER II

## BACKGROUND

This section introduces the technical concepts necessary to understand the experimental comparison.

### Reinforcement Learning and PPO

In reinforcement learning, an agent interacts with an environment modeled as a Markov Decision Process (MDP), defined by states S, actions A, transition dynamics T, and a reward function R. The agent learns a policy $\pi$ that maximizes expected cumulative reward. Proximal Policy Optimization (PPO) is a policy gradient algorithm that stabilizes training by constraining the size of policy updates at each step [9]. PPO is widely used for discrete control tasks and serves as the learning algorithm for all experiments in this work.

### Linear Temporal Logic (LTL)

Linear Temporal Logic is a formal specification language used to express properties over sequences of states [1]. LTL formulas are built from atomic propositions and temporal operators, including:

- $F(\varphi)$ – "eventually $\varphi$" – $\varphi$ must hold at some future timestep
- $G(\varphi)$ – "globally $\varphi$" – $\varphi$ must hold at all future timesteps
- $\varphi \wedge \Psi$ – "$\varphi$ and $\Psi$" – both must hold simultaneously

Infinite-horizon LTL specifications are enforced by converting the formula into a finite automaton that tracks the agent's progress through specification states. Reward shaping is applied based on automaton transitions, providing sparse feedback when the agent reaches accepting states.

## Truncated Linear Temporal Logic (TLTL) and Robustness

Truncated Linear Temporal Logic (TLTL) is a finite-horizon specification language that evaluates temporal objectives over bounded time intervals [2]. Rather than discrete automaton states, TLTL uses a robustness measure $p \in [-1,1]$ that quantifies the degree to which a trajectory satisfies or violates the specification. A positive robustness value indicates satisfaction, negative indicates violation, and the magnitude reflects how strongly the specification is satisfied or violated. This robustness value can be used directly as a dense reward signal at each timestep, providing continuous feedback throughout the episode rather than only at acceptance.

## MiniGrid Environments

MiniGrid provides lightweight, discrete grid-world environments for reinforcement learning research [see Figure 2]. Agents observe a partial view of the grid and must navigate to complete tasks. Three environments of increasing complexity are used in this work: MiniGrid-Empty-8x8-v0 for simple reachability, MiniGrid-LavaGapS5-v0 for safety-constrained navigation, and MiniGrid-DoorKey-8x8-v0 for ordered sequential subtask completion.

## Related Works

Automaton-based LTL approaches translate specifications into product MDPs and use acceptance conditions to guide the learning process. Bozkurt et al. propose a foundational framework that synthesizes policies satisfying LTL specifications in unknown stochastic environments, shaping rewards according to automaton progress while providing convergence guarantees [5]. However, these methods often suffer from

sparse rewards and slow convergence due to the discrete nature of automaton acceptance signals.

To address convergence challenges, Jiang et al. propose temporal logic-based reward shaping techniques that provide denser feedback and accelerate learning without sacrificing policy optimality [6]. Traditional automaton-based methods have also been extended to continuous state and action spaces using deep RL [7]. While these advances improve practicality, they continue to rely on automaton constructions and infinite-horizon objectives.

Robustness-based approaches avoid explicit automaton constructions by incorporating temporal structure directly into the reward signal. Li et al. introduce TLTL as a specification language for finite-horizon tasks, encoding temporal objectives as robustness measures that enable continuous reward signals [2]. Similarly, Balakrishnan and Deshmukh use Signal Temporal Logic (STL) to generate locally shaped rewards that guide deep RL while remaining agnostic to the underlying algorithm [3].

Compositional approaches decompose complex temporal specifications into smaller subtasks solved independently. The DIRL framework interleaves high-level planning with RL by learning policies for individual subtasks and composing them to satisfy the overall specification, improving scalability to more complex tasks [8]. Similarly, Wang and Zhu propose deductive synthesis methods for infinite-horizon tasks that leverage logical structure for improved policy learning [10].

Despite these advances, most existing methods focus on either infinite-horizon or task-specific finite-horizon formulations. Direct empirical comparisons between the two

approaches across identical environments and learning algorithms remain limited, motivating the controlled evaluation presented in this work.

# CHAPTER III

# METHODS

## 3.1 Experimental Setup

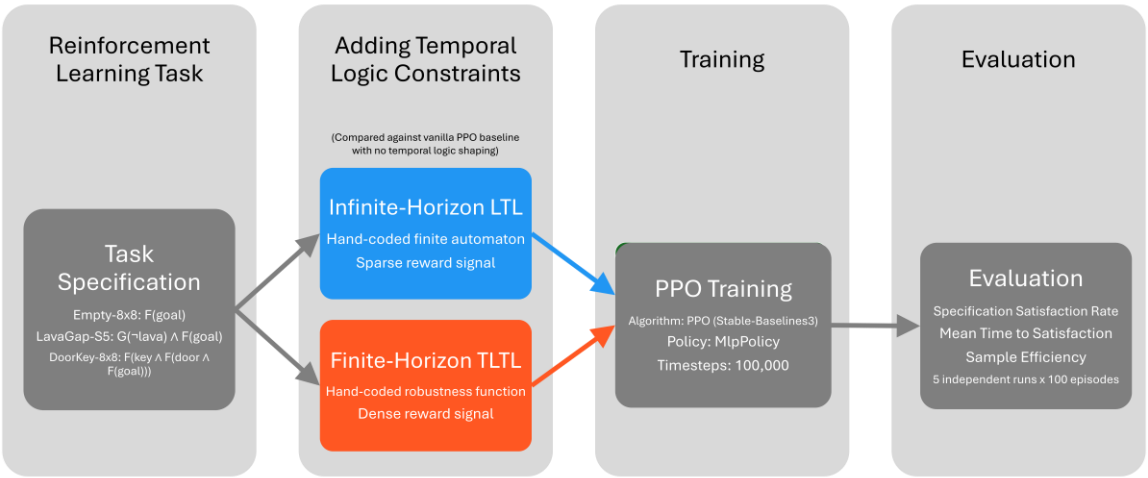


**Figure 1: Overview of experimental pipeline.**

This figure illustrates the overall workflow of the study. The process begins with encoding temporal logic specifications, followed by integrating either infinite-horizon automaton constructions or finite-horizon robustness rewards into a PPO training loop. Agents are then trained in multiple MiniGrid environments and evaluated using standardized performance metrics including specification satisfaction probability, mean time to satisfaction, and sample efficiency. This diagram clarifies how both temporal logic formulations share an identical learning algorithm to ensure a fair comparison.

All experiments were conducted using Gymnasium MiniGrid environments, which provide lightweight, discrete grid-world tasks that support rapid training and clear success/failure evaluation. Three benchmark environments of increasing complexity were used (see Figure 2):

- MiniGrid-Empty-8x8-v0: A simple 8x8 grid in which the agent must navigate to a goal tile. This environment evaluates basic reachability under the specification F(goal).
- MiniGrid-LavaGapS5-v0: A 5x5 grid in which the agent must reach the goal while avoiding lava tiles. This environment evaluates safety-constrained navigation under the specification G(¬lava) ∧ F(goal).
- MiniGrid-DoorKey-8x8-v0: An 8x8 grid requiring the agent to retrieve a key, unlock a door, and then reach the goal. This environment evaluates ordered sequential subtask completion under the specification F(key ∧ F(door ∧ F(goal))).

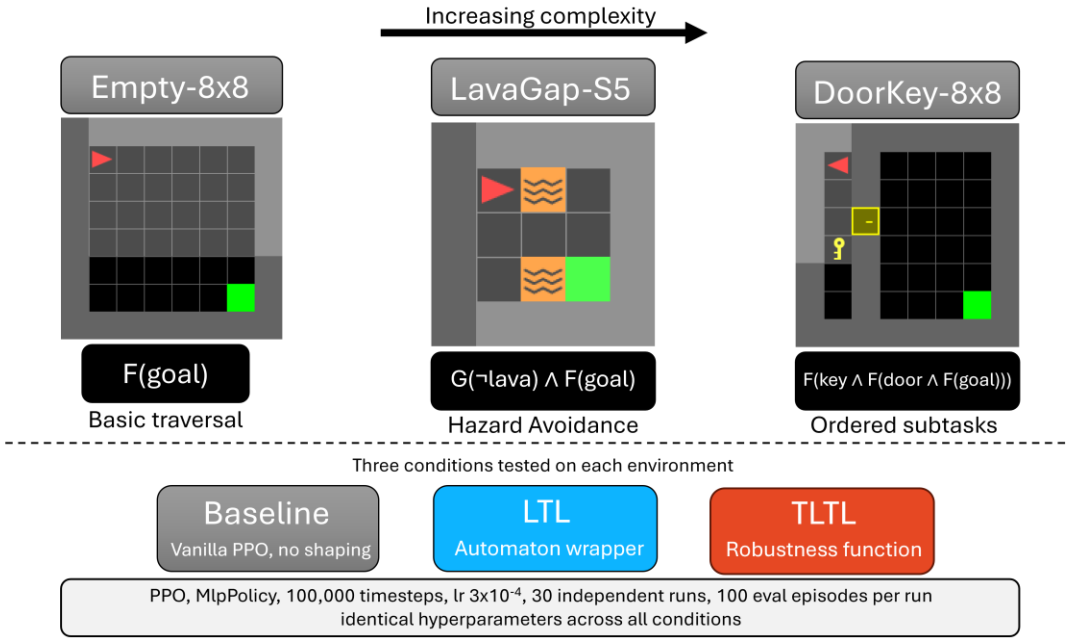


**Figure 2: MiniGrid environments and corresponding temporal specifications.**

Figure 2 presents screenshots of the three Gymnasium MiniGrid tasks used for evaluation: simple goal reachability, hazard avoidance with lava tiles, and a sequential door-key task requiring ordered objectives. Each environment is paired with its corresponding temporal logic specification illustrating how reachability, safety, and sequential constraints are formally encoded. These tasks increase in complexity and demonstrate how different specification structures stress the learning methods.

All environments use partial observability, where agents observe only a limited field of view rather than the full grid. This makes the tasks more challenging and realistic.

**3.2 Baseline RL Agent**

A baseline reinforcement learning agent was implemented using the Proximal Policy Optimization (PPO) algorithm via Stable-Baselines3 to ensure reliability and reproducibility. The training and evaluation pipeline was constructed including environment integration, logging of performance metrics, and standardized hyperparameters. A FlatObsWrapper was applied to flatten the MiniGrid observation space into a single vector suitable for an MLP policy. All conditions used identical hyperparameters: 100,000 training timesteps, learning rate $3*10^{-4}$, n_steps 2,048, batch size 64, and 10 epochs per update, to ensure that only the temporal logic formulation varies across conditions, enabling fair comparison.

**3.3 Infinite-Horizon LTL Implementation**

Infinite-horizon LTL specifications were integrated into the PPO framework as hand-coded finite automaton environment wrappers. Rather than using existing automaton libraries, automata were implemented directly in Python to maintain transparency and avoid dependency issues. Each wrapper tracks the agent's progress through discrete automaton states Q_running, Q_accept, and Q_dead, providing +1 reward upon reaching Q_accept and -1 upon reaching Q_dead. The three environment-specific automata encode the following behaviors:

- Empty-8x8: F(goal) | single transition from Q_running to Q_accept on goal contact
- LavaGap-S5: G(¬lava) ∧ F(goal) | transition to Q_dead on lava contact, Q_accept on goal contact

- DoorKey-8x8: F(key ∧ F(door ∧ F(goal))) | four-state automaton tracking sequential subtask completion through states Q_no_key → Q_has_key → Q_door_open → Q_accept, with +1 reward on each forward transition

**3.4 Finite-Horizon TLTL Implementation**

Finite-horizon TLTL specifications were implemented using robustness-based reward shaping. Temporal objectives were encoded as horizon-bounded formulas and evaluated using robustness measures $p \in [-1,1]$ that quantify the degree to which a trajectory satisfies the specification. Unlike the automaton-based infinite-horizon approach, this method avoids explicit product MDP construction and instead provides continuous feedback directly from the robustness value. The shaped reward at each timestep is the change in robustness $\Delta p$, with a terminal bonus of +1.0 on task completion. The three environment-specific robustness functions are:

- Empty-8x8: p = normalized negative distance to goal, rewarding incremental progress toward the goal at every step
- LavaGap-S5: p = min(lava_safety_margin, goal_progress), using the conjunction operator from TLTL semantics to jointly enforce safety and reachability
- DoorKey-8x8: p = phased continuous value tracking sequential subtask progress across three phases (0.0-0.33 for key retrieval, 0.33-0.67 for door opening, 0.67-1.0 for goal reaching)

**3.5 Evaluation Framework**

An evaluation framework was developed to ensure consistent comparison across methods. Each of the nine configurations (3 environments x 3 methods) were trained across 30 independent runs without fixed seeds to assess consistency across different

initializations. Each trained model was evaluated over 100 episodes using stochastic policy evaluation. Three metrics were computed:

- Specification satisfaction rate: the fraction of evaluation episodes in which the agent successfully completed the full task specification
- Mean time to satisfaction: mean episode length across successful episodes only, measuring task completion efficiency
- Sample efficiency: number of training timesteps required to achieve an 80% reduction in mean episode length, used to determine learning speed

Training statistics were logged automatically at each episode and stored as CSV files for subsequent analysis. Statistical comparisons between methods were performed using Welch's t-test on per-run satisfaction rates across the 30 runs, with results reported as mean ± standard deviation.

# CHAPTER IV

# RESULTS

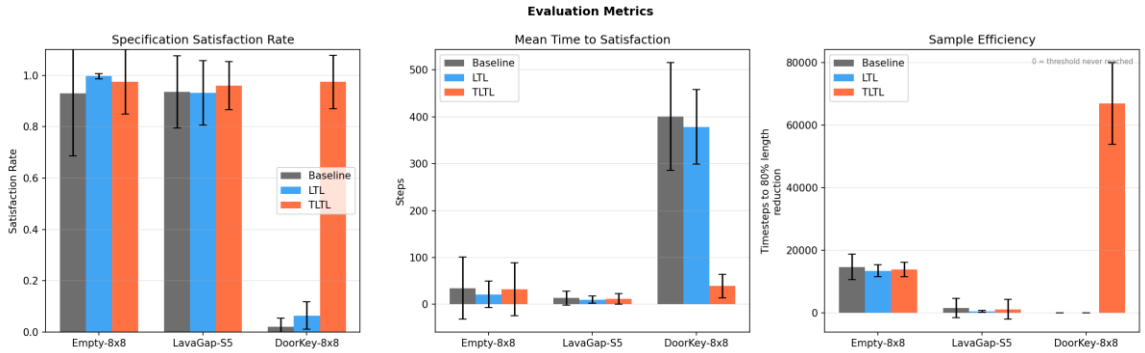


**Figure 3: Bar plots of evaluation metrics (specification satisfaction rate, mean time to satisfaction, and sample efficiency) across all runs.**

Figure 3 presents bar plots of the three evaluation metrics for every run. TLTL performs measurably better on the DoorKey-8x8 task comparably to the other methods, but similarly on Empty-8x8 and LavaGap-S5. The results show the benefits of using finite-horizon temporal logic on ordered subtask problems.

## 4.1 Specification Satisfaction Rate

| Environment | **Baseline** | **LTL** | **TLTL** |
|---|---|---|---|
| **Empty-8x8** | 0.930 ± 0.244 | 0.997 ± 0.010 | 0.976 ± 0.126 |
| **LavaGap-S5** | 0.936 ± 0.141 | 0.932 ± 0.126 | 0.961 ± 0.094 |
| **DoorKey-8x8** | 0.020 ± 0.034 | 0.064 ± 0.054 | 0.974 ± 0.104 |

**Table 1: Mean ± standard deviation of specification satisfaction rate across 30 runs x 100 episodes.**

Table 1 summarizes specification satisfaction rates across all environments and methods. On simple tasks, all three methods performed comparably with no statistically significant differences. On the sequential DoorKey-8x8 task, TLTL dramatically outperformed both alternatives.

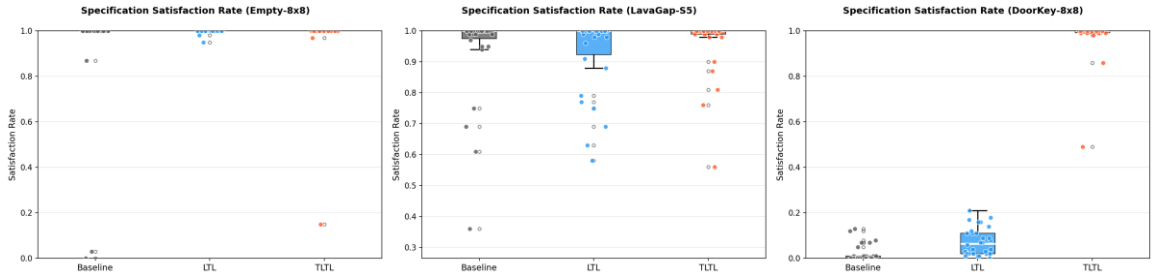


**Figure 4: Specification satisfaction rate boxplots.**

No significant differences between methods on Empty-8x8 and LavaGap-S5 ($p>0.05$). On DoorKey-8x8, all pairwise comparisons were significant ($p<0.001$), with TLTL achieving nearly perfect satisfaction while Baseline and LTL largely failed.

On Empty-8x8, all three methods achieved high satisfaction rates with no significant differences between any pair (all $p>0.05$ using Welch's t-test). LTL performed most consistently with the lowest variance ($0.997 \pm 0.010$), while Baseline showed notably higher variance ($0.930 \pm 0.244$), suggesting occasional training instability.

On LavaGap-S5, all three methods again performed comparably with no significant differences (all $p>0.05$). TLTL achieved the highest mean satisfaction rate ($0.961 \pm 0.094$) with the lowest variance, while LTL showed the most variance across runs ($0.932 \pm 0.126$).

On DoorKey-8x8, the results diverged dramatically. The Baseline agent largely failed to learn the task, achieving only 2.0% satisfaction across 30 runs. LTL showed marginal improvement at 6.4%, while TLTL achieved 97.4% satisfaction. All pairwise comparisons were highly significant ($p<0.0001$), including Baseline vs LTL ($t=-4.286$, $p<0.0001$), Baseline vs TLTL ($t=-51.281$, $p<0.0001$), and LTL vs TLTL ($t=-43.802$, $p<0.0001$). These results are summarized in Figure 4.

## 4.2 Mean Time to Satisfaction

| Environment | **Baseline** | **LTL** | **TLTL** |
|---|---|---|---|
| **Empty-8x8** | 34.5 ± 66.1 steps | 21.4 ± 28.5 steps | 32.0 ± 56.6 steps |
| **LavaGap-S5** | 13.4 ± 14.8 steps | 10.1 ± 7.5 steps | 11.7 ± 11.2 steps |
| **DoorKey-8x8** | 400.6 ± 114.8 steps | 378.5 ± 79.2 steps | 39.5 ± 25.1 steps |

**Table 2: Mean ± standard deviation of episode length across successful episodes only.**

Table 2 reports mean episode length across successful episodes only, measuring how efficiently each method completes the task once learned.

On Empty-8x8 and LavaGap-S5, all three methods completed successful episodes in a similar number of steps, with high standard deviations reflecting variance across runs. On DoorKey-8x8, TLTL completed successful episodes in 39.5 ± 25.1 steps on average, compared to 400.6 ± 114.8 steps for Baseline and 378.5 ± 79.2 steps for LTL. This result is particularly notable given that the DoorKey environment has a maximum episode length of 640 steps, meaning Baseline and LTL agents that did solve the task were doing so only near the episode time limit, while TLTL agents solved it efficiently.

## 4.3 Sample Efficiency

| Environment | **Baseline** | **LTL** | **TLTL** |
|---|---|---|---|
| **Empty-8x8** | 14,680 ± 4,057 | 13,464 ± 1882 | 13,867 ± 2,268 |
| **LavaGap-S5** | 1,542 ± 3,078 | 490 ± 279 | 1148 ± 3,158 |
| **DoorKey-8x8** | N/A | N/A | 66,863 ± 13,063 |

**Table 3: Mean ± standard deviation of timesteps to 80% episode length reduction.**

Table 3 reports the number of training timesteps required to achieve an 80% reduction in mean episode length, used to measure learning speed. A value of N/A indicates the threshold was never reached within 100,000 timesteps.

On Empty-8x8, all three methods reached the threshold at a similar number of timesteps (~13,000–14,700), consistent with their comparable satisfaction rates. On LavaGap-S5, LTL appeared most sample efficient (490 ± 279 timesteps), however this result should be interpreted with caution because the metric measures episode length reduction rather than task success, and LTL agents on LavaGap-S5 may reduce episode length by failing quickly on lava rather than solving the task efficiently. On DoorKey-8x8, only TLTL reached the threshold, doing so at 66,863 ± 13,063 timesteps. Baseline and LTL never achieved sufficient episode length reduction within the 100,000 timestep budget.

## 4.4 Learning Curves

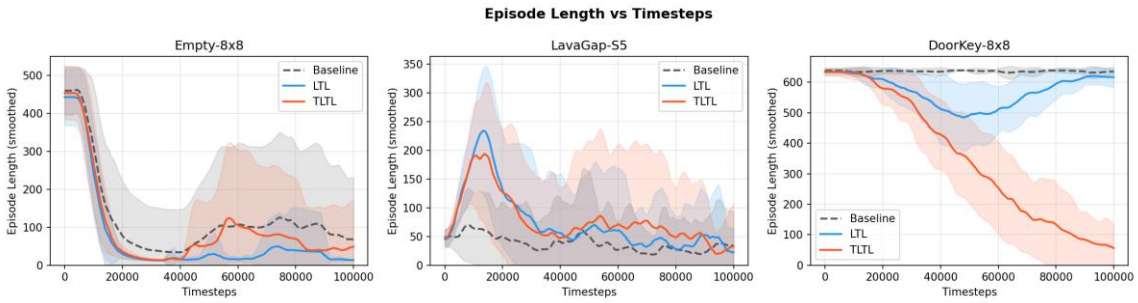


**Figure 5: Episode length learning curves averaged over 30 runs.**

Figure 5 shows episode length learning curves averaged across 30 runs with ± one standard deviation shaded. On Empty-8x8 and LavaGap-S5, all three methods converge to short episode lengths within the first 20,000-30,000 timesteps, with overlapping confidence bands confirming no meaningful difference in learning dynamics. On DoorKey-8x8, the divergence between methods is clear. Baseline and LTL episode lengths remain near the maximum of 640 steps throughout training, while TLTL episode length steadily decreases, reaching approximately 40 steps by 100,000 timesteps. The

narrower confidence band for TLTL on DoorKey-8x8 relative to Baseline and LTL indicates more consistent learning across runs.

# CHAPTER V

# DISCUSSION

This study compares three reward shaping approaches for reinforcement learning across environments of increasing complexity: a vanilla PPO baseline, infinite-horizon LTL with automaton-based reward shaping, and finite-horizon TLTL with robustness-based reward shaping. The results reveal a clear pattern where the choice of temporal logic formulation has little impact on simple tasks but becomes critically important as task complexity increases.

## 5.1 Simple Tasks

On Empty-8x8 and LavaGap-S5, all three methods achieved comparable specification satisfaction rates with no statistically significant differences. This result suggests that for tasks with simple reachability or single safety constraints, the additional structure provided by temporal logic reward shaping does not meaningfully improve over vanilla PPO. The default sparse reward signal is sufficient to guide learning when the task structure is simple enough for the agent to discover successful behaviors through exploration alone.

Notably, the Baseline agent showed higher variance than LTL and TLTL on both simple environments, suggesting that temporal logic shaping may improve training consistency even when it does not improve mean performance. This is consistent with the intuition that structured reward signals provide more informative feedback during exploration, reducing the likelihood of training runs that fail to converge.

## 5.2 Sequential Subtask Performance

The DoorKey-8x8 results represent the primary finding of this work. The Baseline agent largely failed to learn the task, achieving only 2.0% satisfaction across 30 runs. This is expected because the task requires completing three ordered subtasks (retrieve key, open door, reach goal) under sparse reward, making it extremely unlikely that random exploration will produce a complete successful episode to learn from. LTL showed marginal improvement at 6.4%, suggesting that automaton-based reward shaping provides some benefit by rewarding individual subtask transitions. However, this improvement was modest and highly variable across runs.

TLTL achieved 97.4% satisfaction with a mean solve time of 39.5 steps, dramatically outperforming both alternatives. This result supports the hypothesis that dense continuous reward signals are significantly more effective than sparse automaton-based rewards for ordered subtask problems. The robustness function provides meaningful gradient-like feedback at every timestep, guiding the agent toward the next subtask even when it has not yet completed the current one. This contrasts with LTL's sparse reward structure, which provides no feedback until an automaton state transition occurs.

### 5.3 Training Stability

The learning curve analysis reveals the additional advantage of TLTL that it is more consistent across runs. On DoorKey-8x8, TLTL's confidence band narrows considerably over training, indicating that most runs converge to similar policies. Baseline and LTL maintain wide confidence bands throughout, reflecting high run-to-run variance and frequent training failures. This stability advantage is practically important; a

method that reliably converges is more useful in real applications than one that occasionally achieves high performance but frequently fails.

The high variance observed for LTL on Empty-8x8 is also worth noting. In several runs, the LTL agent failed to learn the task entirely, producing a satisfaction rate of 0.0. This instability on a simple task is unexpected and may reflect sensitivity to initial conditions under sparse reward shaping. Future work should investigate whether this instability persists across longer training horizons.

**5.4 Limitations**

Several limitations of this work should be acknowledged. First, random seeds were not fixed across runs, meaning exact replication of individual runs is not possible. While this was intentional to assess consistency across different initializations, it limits strict reproducibility. Second, temporal logic specifications were hand-coded rather than compiled from formal specifications using automated tools such as SPOT. While this approach is transparent and correct for the three environments used, it does not scale to arbitrary specifications and introduces the possibility of implementation error. Third, all experiments used a fixed training budget of 100,000 timesteps. It is possible that LTL would achieve higher satisfaction rates given longer training, and the relative advantage of TLTL may diminish with more timesteps. Finally, results are specific to MiniGrid environments and may not generalize to continuous state spaces or more complex real-world tasks.

Despite these limitations, the consistency of results across 30 independent runs and the magnitude of the performance gap on DoorKey-8x8 provide strong evidence that finite-horizon robustness-based reward shaping is substantially more effective than

automaton-based approaches for sequential subtask problems within the experimental conditions studied here. Extending this comparison to continuous environments and longer training horizons represents the most important direction for future work.

# CHAPTER VI

# CONCLUSION

This work presented an empirical comparison of infinite-horizon LTL and finite-horizon TLTL reward shaping approaches for reinforcement learning across three MiniGrid environments of increasing complexity. Both approaches were integrated into an identical PPO framework to isolate the effect of temporal logic formulation, and performance was evaluated using specification satisfaction probability, mean time to satisfaction, and sample efficiency across 30 independent runs.

Results demonstrate that on simple reachability and safety-constrained tasks, all three methods (vanilla PPO baseline, LTL, and TLTL) perform comparably, with no statistically significant differences in satisfaction rates. On the sequential DoorKey-8x8 task, TLTL significantly outperformed both alternatives, achieving 97.4% satisfaction compared to 6.4% for LTL and 2.0% for the baseline ($p<0.0001$). TLTL also completed successful episodes much faster, solving DoorKey-8x8 in a mean of 39.5 steps compared to over 400 steps for Baseline and LTL. These results confirm that finite-horizon robustness-based reward shaping provides significantly more effective learning signals for ordered subtask problems than sparse automaton-based rewards.

These findings have practical implications for the design of temporally constrained RL systems. When tasks involve ordered subtask completion or complex sequential constraints, finite-horizon robustness-based formulations should be preferred over infinite-horizon automaton-based approaches due to their superior sample efficiency and training stability. When tasks are simpler, the choice of formulation matters less and the added implementation complexity of temporal logic shaping may not be warranted.

Future work should investigate several open questions raised by this study. First, longer training horizons may close the performance gap between LTL and TLTL on complex tasks. This would help in determining whether TLTL's advantage is primarily one of sample efficiency or asymptotic performance is an important distinction. Second, systematically varying hyperparameters such as learning rate or optimizer could help to fully test their effect on the dominance shown by TLTL on complex tasks. Third, evaluating these methods in continuous state spaces and more complex environments would strengthen the generalizability of these findings.

# BIBLIOGRAPHY

[1] Rozier, K. Y. (2011). Linear Temporal Logic Symbolic Model Checking. Computer Science Review, 5(2), 163–203. https://doi.org/10.1016/j.cosrev.2010.06.002

[2] Li, X., Vasile, C.-I., & Belta, C. (2017). Reinforcement learning with temporal logic rewards. 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 3834–3839. https://doi.org/10.1109/IROS.2017.8206234

[3] Balakrishnan, A., & Deshmukh, J. V. (2019). Structured Reward Shaping using Signal Temporal Logic specifications. 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 3481–3486. https://doi.org/10.1109/IROS40897.2019.8968254

[4] Lennartson, B., & Jia, Q.-S. (2020). Reinforcement Learning with Temporal Logic Constraints. IFAC-PapersOnLine, 53(4), 485–492. https://doi.org/10.1016/j.ifacol.2021.04.044

[5] Bozkurt, A. K., Wang, Y., Zavlanos, M. M., & Pajic, M. (2020). Control Synthesis from Linear Temporal Logic Specifications using Model-Free Reinforcement Learning. 2020 IEEE International Conference on Robotics and Automation (ICRA), 10349–10355. https://doi.org/10.1109/ICRA40945.2020.9196796

[6] Jiang, Y., Bharadwaj, S., Wu, B., Shah, R., Topcu, U., & Stone, P. (2021). Temporal-Logic Based Reward Shaping for Continuing Reinforcement Learning Tasks. Proceedings of the AAAI Conference on Artificial Intelligence, 35(9), 7995–8003. https://doi.org/10.1609/aaai.v35i9.16975

[7] Cai, M., Hasanbeig, M., Xiao, S., Abate, A., & Kan, Z. (2021). Modular Deep Reinforcement Learning for Continuous Motion Planning With Temporal Logic. IEEE Robotics and Automation Letters, 6(4), 7973–7980. https://doi.org/10.1109/LRA.2021.3101544

[8] Jothimurugan, K., Bansal, S., Bastani, O., & Alur, R. (2021). Compositional Reinforcement Learning from Logical Specifications (arXiv:2106.13906). arXiv. https://doi.org/10.48550/arXiv.2106.13906

[9] Shakya, A. K., Pillai, G., & Chakrabarty, S. (2023). Reinforcement learning algorithms: A brief survey. Expert Systems with Applications, 231, 120495. https://doi.org/10.1016/j.eswa.2023.120495

[10] Wang, Y., & Zhu, H. (2025). Deductive Synthesis of Reinforcement Learning Agents for Infinite Horizon Tasks. In R. Piskac & Z. Rakamarić (Eds.), Computer Aided Verification (Vol. 15934, pp. 79–103). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-98685-7_4